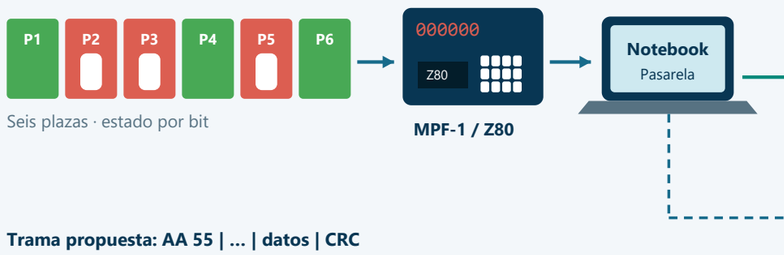
Estacionamiento inteligente UCU

Instalación y prueba con datos simulados · Lámina 1 de 2 · Scripts íntegros incluidos en el ZIP

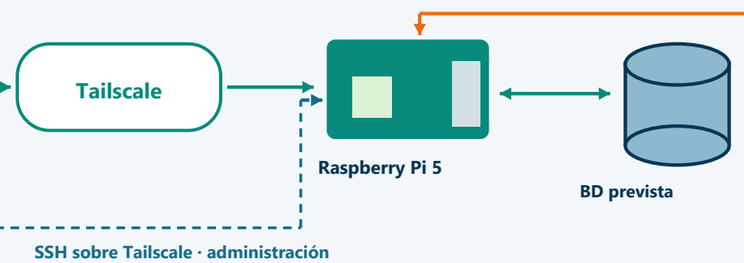
A3 HORIZONTAL · 420 × 297 mm

Actualizado con las pruebas del 21/10/2026

EN CLASE · WINDOWS



ACCESO PRIVADO · RPi5



CONSULTA PÚBLICA · APK



1 Sistema y primer acceso

DOCENTE · IMAGER Y POWERSHELL

En Raspberry Pi Imager: **RPi5**, Raspberry Pi OS Lite **64 bits**, usuario **ucu2026**, nombre **ucu2026**, red configurada y **SSH con contraseña**. Grabar una microSD nueva o respaldar sus archivos antes de sobrescribirla.

```
Start-Process 'https://tailscale.com/download/windows'
ping -4 ucu2026.local
ssh ucu2026@ucu2026.local
```

Instalar Tailscale en la PC e iniciar sesión con la identidad autorizada. El primer SSH local requiere la misma LAN. Si falta el cliente SSH de Windows, instalarlo con TI. Al ver **ucu2026@UCU2026**, continuar en Bash.

2 Preparar RPi5, SSH y Tailscale

RPi5 · BASH · DOCENTE ucu2026

```
hostnamectl
sudo apt-get update
sudo apt-get install -y curl ca-certificates openssh-server python3-venv
sudo systemctl enable --now ssh
curl -fsSL https://tailscale.com/install.sh | sh
sudo systemctl enable --now tailscaled
sudo tailscale up
tailscale status
tailscale ip -4
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Abrir el enlace de Tailscale y autorizar el equipo. Si el instalador agotó su espera de 180 s, repetir **sudo tailscale up**. IP comprobada: **100.72.228.96**. Una reinstalación puede asignar otra IP.

Huella ED25519 de esta instalación:
`SHA256:JSR/i/U6rb/MehNms31PXn/xe+pHOgaZj0ANx73hrgs`

3 Crear los cuatro grupos

RPi5 · BASH · DOCENTE ucu2026

```
for n in 1 2 3 4; do
 cuenta="grupo$n"
 if ! getent passwd "$cuenta" >/dev/null; then
  sudo useradd --create-home --user-group \
   --shell /bin/bash "$cuenta"
  sudo passwd "$cuenta"
 fi
done
getent passwd grupo1 grupo2 grupo3 grupo4
```

Cada cuenta tiene su contraseña y home **/home/grupoN**. El instalador crea las cuentas que faltan. No se incluyen contraseñas ni tokens en este material.

Cada PC también necesita autorización de Tailscale. Compartir la RPi con la identidad del estudiante o incorporarla a la red del curso; las políticas deben permitir SSH/TCP 22. La cuenta Linux no concede ese acceso.

4 Cargar el paquete y ejecutar

PC POWERSHELL → RPi5 BASH

```
$zip = "$env:USERPROFILE\Downloads\" + `
 'Estacionamiento_UCU_Paquete_Completo.zip'
$dir = "$env:USERPROFILE\Downloads\Estacionamiento_UCU"
Expand-Archive -LiteralPath $zip -DestinationPath $dir
scp -r "$dir\rpi" ucu2026@ucu2026.local:
ssh ucu2026@ucu2026.local
```

```
bash ~/rpi/instalar_estacionamiento_ucu.sh
```

Ruta probada: el docente pegó el instalador en Bash y ejecutó **bash ~/instalar_estacionamiento_ucu.sh**. ZIP/SCP es una alternativa preparada para distribuir los mismos archivos; esa transferencia no se ejecutó en el ensayo.

Después de instalar, cada estudiante entra desde PowerShell con `ssh grupo1@100.72.228.96`, adaptando el usuario. Los pasos 5–8 desglosan lo que automatiza el instalador.

5 Archivos y entorno por grupo

RPi5 · BASH · EJEMPLO grupo1

El instalador genera el servidor y requisitos en **~/PI2_instalacion**. Para cada grupo prepara su carpeta y ejecuta Python con la cuenta correspondiente:

```
g=grupo1
base="/home/$g/PI2/estacionamiento"
sudo install -d -o "$g" -g "$g" -m 0750 \
 "/home/$g/PI2" "$base"
sudo install -o "$g" -g "$g" -m 0640 \
 ~/PI2_instalacion/servidor_estacionamiento.py "$base/"
sudo install -o "$g" -g "$g" -m 0640 \
 ~/PI2_instalacion/requirements.txt "$base/"
sudo -H -u "$g" bash -c '
 cd "$1"
 python3 -m venv .venv
 .venv/bin/python -m pip install -r requirements.txt
' bash "$base"
```

Repetir con grupo2–grupo4. Puertos: **5001, 5002, 5003, 5004**. Seis plazas por desarrollo; las cuatro API no representan 24 plazas físicas.

6 Instalar cloudflared si falta

RPi5 · BASH · DOCENTE ucu2026

```
CF_KEY='/usr/share/keyrings/cloudflare-main.gpg'
CF_REPO='https://pkg.cloudflare.com/cloudflared'
sudo mkdir -p --mode=0755 /usr/share/keyrings
curl -fsSL https://pkg.cloudflare.com/cloudflare-main.gpg \
 | sudo tee "$CF_KEY" >/dev/null
sudo chmod 0644 "$CF_KEY"
printf 'deb [signed-by=%s] %s any main\n' \
 "$CF_KEY" "$CF_REPO" | sudo tee \
 /etc/apt/sources.list.d/cloudflared.list
sudo apt-get update
sudo apt-get install -y cloudflared
cloudflared --version
```

Versiones observadas: Debian 13 arm64, Python 3.13.5, Tailscale 1.104.2, cloudflared 2026.10.2, Flask 3.1.3 y Waitress 3.0.2. Los requisitos aceptan versiones compatibles.

CUENTAS DEL CURSO
grupo1 · grupo2 · grupo3 · grupo4

PUERTOS DE LAS API
5001 · 5002 · 5003 · 5004

TAILSCALE · DIRECCIÓN COMPROBADA
100.72.228.96

Alcance: esta instalación simula las lecturas en la RPi5 cada 10 s, con vigencia de 20 s e historial en memoria. MPF-1, sensores, pasarela, almacenamiento persistente y coordenadas relevadas siguen pendientes. Código Python, instalador completo, pruebas y plantillas están incluidos en el ZIP.

Arranque automático y prueba de extremo a extremo

Lámina 2 de 2 · Cuatro API y cuatro túneles comprobados después de reiniciar · 21/10/2026

SERVICIOS Y REINICIO

7 Instalar las dos plantillas

RPi5 · BASH · DOCENTE ucu2026

El instalador escribe ambas unidades. Si se usa el paquete copiado en **~/rpi**, estos comandos instalan sus plantillas completas:

```
sudo install -m 0644 ~/rpi/estacionamiento@.service \
 /etc/systemd/system/estacionamiento@.service
sudo install -m 0644 ~/rpi/estacionamiento-tunel@.service \
 /etc/systemd/system/estacionamiento-tunel@.service
sudo systemctl daemon-reload
```

Configuración dentro de los archivos (las continuaciones con \ son válidas en systemd):

```
# estacionamiento@.service
User=grupo%i
WorkingDirectory=/home/grupo%i/PI2/estacionamiento
ExecStart=/home/grupo%i/PI2/estacionamiento/.venv/bin/python \
 /home/grupo%i/PI2/estacionamiento/servidor_estacionamiento.py \
 --grupo %i --host 0.0.0.0
```

```
# estacionamiento-tunel@.service
User=grupo%i
ExecStart=/usr/bin/cloudflared tunnel --no-autoupdate \
 --protocol http2 --url http://127.0.0.1:500%i
```

Las unidades completas incluyen grupo de ejecución, directorio, dependencias, **Restart=on-failure** y habilitación al iniciar. La API escucha en LAN/Tailscale; Cloudflare consulta el origen local 127.0.0.1.

8 Activar y probar el reinicio

RPi5 · BASH · CORTA LA SESIÓN SSH

```
for n in 1 2 3 4; do
 sudo systemctl enable "estacionamiento@$n" \
  "estacionamiento-tunel@$n"
 sudo systemctl restart "estacionamiento@$n" \
  "estacionamiento-tunel@$n"
done
sudo reboot
```

Al reiniciar cambian las URL de prueba. También se borra el historial en memoria. Cuando vuelva a arrancar, reconectar desde PowerShell con `ssh ucu2026@100.72.228.96` y seguir el paso 9.

CUATRO DESARROLLOS · SEIS PLAZAS POR API

grupo1 → 5001 → API y túnel comprobados
grupo2 → 5002 → API y túnel comprobados
grupo3 → 5003 → API y túnel comprobados
grupo4 → 5004 → API y túnel comprobados
Las cuatro pruebas locales y HTTPS aprobaron.

VERIFICACIÓN Y APK

9 Comprobar después del reinicio

RPi5 · BASH · DOCENTE ucu2026

```
bash ~/rpi/verificar_despues_reinicio.sh
cat ~/PI2_instalacion/direcciones_apk.txt
```

El script muestra sistema/IP, comprueba **active y enabled** de ocho unidades, prueba cuatro API locales y cuatro HTTPS, y valida grupo, seis identificadores únicos, estados y lectura vigente. Lee las URL del arranque actual y reemplaza el archivo solo si todo aprueba.

```
=== VERIFICACION DESPUES DEL REINICIO: OK ===
```

Direcciones verificadas el 21/10, después de reiniciar:

```
1 https://goals-acts-changed-sign.trycloudflare.com
2 https://compact-earlier-previous-appear.trycloudflare.com
3 https://chicken-casa-walter-charlie.trycloudflare.com
4 https://frog-resulting-enrolled-arts.trycloudflare.com
```

Son direcciones temporales. Después de otro reinicio o de recrear un túnel, ejecutar de nuevo el verificador y configurar la URL nueva en la APK. Un archivo de direcciones anterior puede estar desactualizado.

10 Configurar y comprobar la APK

TELÉFONO ANDROID · EQUIPO 1 COMO EJEMPLO

1. Instalar **android/Estaciona-Salto-UCU-0.1.3.apk** del paquete si falta; requiere Android 8 o posterior.
2. Abrir **Ajustes** y desactivar **Modo demostración**.
3. En **Dirección del servidor HTTPS**, pegar solo el dominio correspondiente al equipo.
4. Elegir **Equipo 1** y tocar **Guardar y conectar**.
5. En el teléfono, desactivar Wi-Fi y Tailscale, usar datos móviles y comprobar las seis plazas y sus cambios.

La APK se conserva sin recompilar y trae una URL vieja como valor inicial: actualizarla. Desactivar la demostración local no cambia los datos simulados del servidor por datos reales.

El usuario confirmó que la APK funcionó antes del reinicio. **Falta confirmar otra vez el teléfono con la URL nueva**. La app pública no necesita cuenta Linux ni Tailscale.

APK · ÚLTIMA COMPROBACIÓN PENDIENTE

Actualizar en Ajustes la dirección del Equipo 1:
`https://goals-acts-changed-sign.trycloudflare.com`
Equipo 1 · demostración desactivada · Guardar y conectar.
Confirmar en el teléfono después del cambio.

EVIDENCIA Y CONTINUIDAD

11 Diagnóstico y resultados observados

RPi5 · BASH · CONSULTA

```
systemctl list-units --all --no-pager 'estacionamiento*'
sudo ss -ltnp '( sport = :5001 )'
systemctl status estacionamiento@1 --no-pager -l
sudo journalctl -u estacionamiento@1 -n 20 --no-pager
sudo journalctl -b -u estacionamiento-tunel@1 \
 -n 30 --no-pager
```

APROBADO TRAS REINICIAR:
8 servicios active/enabled · 4 API locales OK · 4 HTTPS OK · 6 plazas simuladas por grupo · fecha vigente.
IP Tailscale: **100.72.228.96**. El cambio de Boot ID corrobora el nuevo arranque.

Errores anteriores: Test-NetConnection y $env:USERPROFILE pertenecen a PowerShell. En Bash fallaron. El puerto 5001 anterior estaba sin listener; el servicio actual se comprobó después de reiniciar. El registro completo y las diferencias con la lámina anterior están en **evidencia/Registro_Verificacion_Estacionamiento_UCU.md**.

12 Dirección fija: siguiente etapa

DOCENTE · CLOUDFLARE · NO EJECUTADO EN ESTA PRUEBA

Requiere una cuenta y dominio administrado en Cloudflare. Crear un túnel con nombre en **Networking → Tunnels**, elegir Linux/ARM64 y obtener su token. Revisar si ya existe un conector global antes de instalarlo.

```
read -rs -p 'Token del tunel: ' CF_TUNNEL_TOKEN
printf '\n'
sudo cloudflared service install "$CF_TUNNEL_TOKEN"
unset CF_TUNNEL_TOKEN
sudo systemctl enable --now cloudflared
systemctl is-active cloudflared
```

Crear una ruta pública por grupo en el túnel: hostname propio y servicio **http://127.0.0.1:5001** para grupo1; adaptar a 5002–5004. Probar los dominios y ponerlos en Ajustes de la app. El token se mantiene en el servidor.

Esta etapa **no fue desplegada ni validada**. Para conservar acceso público desde la APK, evitar una autenticación de navegador que la app actual no implemente. Los túneles rápidos no proporcionan un nombre permanente.

SIGUIENTE ETAPA DEL PROYECTO

Dominio fijo y conector permanente.
Lecturas del MPF-1, sensores, serial y CRC.
Pasarela y almacenamiento persistente.
Relevamiento de coordenadas de las seis plazas.
La prueba actual usa datos simulados.

Evidencia: comandos ejecutados por el docente en UCU2026 y salidas revisadas; pruebas locales y HTTPS aprobadas antes/después de reiniciar. APK informada antes del reinicio. **Fuentes oficiales:** Raspberry Pi, Tailscale, pkg.cloudflare.com, Cloudflare Tunnel y systemd. Enlaces, scripts íntegros y límites de validación en LEEME.md.

Guía del ZIP: dónde está cada archivo

Anexo A · Abrir después de las dos láminas originales · Rutas relativas a la carpeta extraída

A3 HORIZONTAL · 420 × 297 mm

Ampliación · 21/10/2026

EMPEZAR POR LEEME.md Y GUIA_DEL_ZIP.md · DESPUÉS ELEGIR INSTALAR O VERIFICAR

En Windows: **Descargas → ZIP → Extraer todo → carpeta nueva**. Entrar donde se ven **rpi, android, laminas y evidencia**. Los textos se abren con Bloc de notas. La carpeta **rpi** se copia a la Raspberry; el archivo **.apk** se transfiere al teléfono.

RUTA EXACTA DENTRO DEL ZIP	PARA QUÉ SIRVE	DÓNDE Y CUÁNDO USARLO
`LEEME.md`	Instrucciones generales, resultados, límites y fuentes.	PC · Leer primero.
`GUIA_DEL_ZIP.md`	Este anexo como texto, con rutas y comandos copiables.	PC · Recorrido completo de las páginas 3–5.
`COMANDOS_PARA_COPIAR.txt`	Bloques de las láminas y de esta ampliación.	Copiar por separado a PowerShell o Bash.
`laminas/Estacionamiento_UCU_Completo_A3.pdf`	PDF ampliado: dos láminas conservadas más tres páginas nuevas.	PC · Consulta e impresión A3.

`laminas/Estacionamiento_UCU_Completo_A3_01.png`
`laminas/Estacionamiento_UCU_Completo_A3_02.png`
`laminas/Estacionamiento_UCU_Completo_A3_03.png`
`laminas/Estacionamiento_UCU_Completo_A3_04.png`
`laminas/Estacionamiento_UCU_Completo_A3_05.png`
	Una imagen por página del PDF. 01–02: láminas originales; 03: mapa; 04: copia e instalación; 05: reinicio y APK.	PC · Consulta visual e impresión.

`rpi/instalar_estacionamiento_ucu.sh`	Instalador completo: cuentas, entornos, API, servicios, túneles y pruebas.	RPi · Bash como ucu2026 · Instalación nueva.
`rpi/verificar_despues_reinicio.sh`	Comprueba el sistema y guarda las URL actuales si todo aprueba.	RPi · Bash como ucu2026 · Ya instalado.
`rpi/servidor_estacionamiento.py`	Código de referencia de la API; simula seis plazas por grupo.	RPi · El servicio ejecuta la copia desplegada.
`rpi/requirements.txt`	Dependencias de Python: Flask y Waitress.	RPi · El instalador las instala en cada .venv.
`rpi/estacionamiento@.service`	Plantilla completa del arranque de las cuatro API.	Configuración systemd · Pasos 5–7 originales.
`rpi/estacionamiento-tunel@.service`	Plantilla completa de los cuatro túneles de Cloudflare.	Configuración systemd · Pasos 5–7 originales.
`rpi/prueba_local.py`	Comprueba las cuatro API locales, historial y separación de grupos.	RPi · Python 3 · Prueba opcional.
`rpi/prueba_publica.py`	Valida un JSON recién descargado: grupo, plazas y fecha.	RPi · Python 3 · Requiere grupo y archivo JSON.
`android/Estaciona-Salto-UCU-0.1.3.apk`	Aplicación Android que consulta la API pública; APK conservada.	Teléfono Android 8 o posterior · Página 5.
`evidencia/Registro_Verificacion_Estacionamiento_UCU.md`	Bitácora de comandos, resultados, correcciones y pendientes.	PC · Evidencia de la prueba realizada.
`SHA256SUMS.txt`	Huellas SHA-256 de todos los demás archivos del paquete.	Texto para integridad · No es un programa.

Cómo leer el paquete: los archivos **.sh** se ejecutan con Bash en la Raspberry. Las plantillas **.service** son configuración. El **.apk** se instala en Android. Los pasos de uso están en las páginas **4 y 5**; las pruebas Python opcionales se explican en **GUIA_DEL_ZIP.md, apartado 9**.

Conservación: las páginas 1–2 mantienen todo su contenido y su numeración original 1/2 y 2/2. Este anexo agrega las páginas 3–5 y un índice para los 20 archivos del paquete actualizado.

Del ZIP a la Raspberry: extracción, copia y ejecución

Anexo B · Continuación del paso 4 original · Elegir la operación según el estado de la Raspberry

A3 HORIZONTAL · 420 × 297 mm

Ampliación · 21/10/2026

PC DE CLASE · WINDOWS / POWERSHELL

A1 Descargar y extraer el paquete

Guardar **Estacionamiento_UCU_Paquete_Completo.zip** en **Descargas**. En el Explorador: seleccionar el ZIP → **Extraer todo** → elegir una carpeta nueva. Abrir la carpeta que contiene **LEEME.md** y **rpi**.

A2 Alternativa: extraer con PowerShell

```
$descargas = Join-Path $env:USERPROFILE 'Downloads'
$zip = Join-Path $descargas `
  'Estacionamiento_UCU_Paquete_Completo.zip'
$carpeta = Join-Path $descargas 'Estacionamiento_UCU'
Expand-Archive -LiteralPath $zip -DestinationPath $carpeta
Get-ChildItem -LiteralPath $carpeta
```

Si ya extrajiste desde el Explorador, asigná **$carpeta** a la ruta real y omití **Expand-Archive**. Para encontrarla, entrar donde está **rpi**, hacer clic en la barra de dirección del Explorador y copiar esa ruta. Si la carpeta propuesta existe, elegí otra nueva. Si el ZIP tiene "(1)" en el nombre, renombralo o corregí **$zip**.

```
Test-Path "$carpeta\rpi\instalar_estacionamiento_ucu.sh"
Test-Path "$carpeta\rpi\verificar_despues_reinicio.sh"
```

Resultado esperado: True en ambas líneas. Si aparece False, revisá si $carpeta apunta a la carpeta correcta o si hay una carpeta intermedia adicional.

A3 Copiar rpi y abrir la sesión SSH

```
scp -r "$carpeta\rpi" ucu2026@ucu2026.local:
ssh ucu2026@ucu2026.local
```

Usar la misma ventana donde se definió **$carpeta**. **scp** copia archivos y **ssh** abre la terminal de la Raspberry. La primera conexión con **ucu2026.local** requiere estar en la misma red local.

Acceso remoto por Tailscale: cuando ya esté autorizado, sustituir **ucu2026.local** en ambas órdenes por la IP vigente. En esta instalación fue **100.72.228.96**. Verificar la clave de host con el docente antes de aceptar una clave nueva.

RASPBERRY PI · BASH / USUARIO ucu2026

B1 Confirmar usuario y archivos copiados

Al ver **ucu2026@UCU2026:~ $**, ya estás en Bash de la Raspberry. El paquete quedó en **/home/ucu2026/rpi**, abreviado **~/rpi**. Comprobar:

```
whoami
ls -l ~/rpi/instalar_estacionamiento_ucu.sh \
  ~/rpi/verificar_despues_reinicio.sh
```

Debe aparecer **ucu2026** y deben existir los dos archivos. Si falta alguno, revisar la extracción y la copia del bloque A3.

B2 Elegir una de estas dos operaciones

Raspberry nueva: instalar. Con sistema, red y SSH preparados, ejecutar como ucu2026 con sudo e Internet:

```
bash ~/rpi/instalar_estacionamiento_ucu.sh
```

El instalador pide sudo y la contraseña de los grupos que falten. Instala los componentes, crea cuatro API y cuatro túneles y realiza pruebas. Si Tailscale pide autorización, abrir su enlace. Revisar cualquier **PENDIENTE**. Los pasos **5–7** originales explican lo que automatiza.

Raspberry ya instalada: verificar. Es la opción para el sistema que ya aprobó la prueba de reinicio:

```
bash ~/rpi/verificar_despues_reinicio.sh
cat ~/PI2_instalacion/direcciones_apk.txt
```

El verificador consulta los servicios y las API; **no los reinicia**. Requiere que el instalador haya creado ~/PI2_instalacion y los ocho servicios. Guarda las URL actuales solamente si todo aprueba.

B3 Continuar con la prueba y la APK

En la página **5**: probar el reinicio si falta, obtener las direcciones vigentes y conectar el teléfono. El instalador contiene una copia incrustada del código y de las plantillas; los archivos sueltos permiten consultar su contenido.

Terminales: $env:USERPROFILE, Test-Path y Expand-Archive pertenecen a PowerShell de Windows. Los comandos con sudo, bash, ls y ~/rpi se ejecutan dentro de la Raspberry. La transferencia ZIP/SCP está preparada; la prueba operativa original usó el instalador pegado en Bash.

Después de instalar: reinicio, APK y ubicación final

Anexo C · Continuación de los pasos 8–11 originales · Usar las URL actuales que devuelve el verificador

REINICIAR Y COMPROBAR

C1 Reiniciar, si falta esta prueba

Lo hace el docente desde Bash. Si el reinicio ya aprobó, continuar en C3 sin repetirlo.

```
sudo reboot
```

C2 Reconectar desde Windows

La sesión SSH se cierra. Cuando la Raspberry vuelva a estar disponible, reconectar desde **PowerShell de Windows**:

```
ssh ucu2026@100.72.228.96
```

Usar la IP Tailscale de la instalación; una reinstalación puede asignar otra.

C3 En Bash: verificar y leer las URL

```
bash ~/rpi/verificar_despues_reinicio.sh
cat ~/PI2_instalacion/direcciones_apk.txt
```

Se revisan **ocho servicios active/enabled**, cuatro API locales y cuatro HTTPS. Cada grupo debe devolver seis plazas simuladas y una lectura vigente.

```
=== VERIFICACION DESPUES DEL REINICIO: OK ===
```

Después del marcador se muestran cuatro líneas **Equipo N: https://...**. La URL cambia al reiniciar o recrear un túnel. Las direcciones impresas en la lámina 2 registran la prueba del 21/10; usar las que devuelve el verificador ahora.

Si la prueba falla: la lista anterior puede estar desactualizada. Consultar el paso 11 original y los mensajes del servicio; corregir el problema y volver a verificar.

Estado documentado: la prueba de reinicio aprobó en los cuatro grupos. La APK se confirmó antes del reinicio; falta comprobarla otra vez con la dirección nueva.

ENCONTRAR Y CONECTAR LA APK

D1 Buscar el archivo en la PC

Dentro de la carpeta extraída, entrar en **android**. Allí está:

```
Estaciona-Salto-UCU-0.1.3.apk
```

D2 Transferir e instalar en el teléfono

Transferir el .apk, por ejemplo por USB, a **Descargas** del teléfono. En Android: **Archivos → Descargas → archivo .apk**. Si lo pide, autorizar la instalación para la aplicación que abre el archivo. Requiere **Android 8 o posterior**.

D3 Configurar la conexión

1. Abrir la app → **Ajustes**.

2. Desactivar **Modo demostración**.

3. En **Dirección del servidor HTTPS**, pegar la URL base actual de su equipo tomada en C3.

4. Elegir **Equipo 1, 2, 3 o 4**, según el grupo.

5. Tocar **Guardar y conectar**.

Qué pegar: solo **https://...trycloudflare.com**. Quitar el prefijo “Equipo 1:” y no agregar /api/estado?grupo=1. La APK incluida conserva una dirección inicial antigua: cambiarla en Ajustes.

D4 Probar el acceso público

Con datos móviles, Wi-Fi y Tailscale desconectados, comprobar las seis plazas y sus cambios. La APK pública no necesita usuario Linux ni Tailscale.

Desactivar la demostración del teléfono conecta la app al servidor. El servidor de esta prueba sigue generando **datos simulados**.

ARCHIVOS CREADOS EN LA RPi

E1 Carpetas de la Raspberry

```
/home/ucu2026/rpi/
```

Paquete copiado desde Windows: scripts y plantillas de referencia.

```
/home/ucu2026/PI2_instalacion/
```

Archivos generados por el instalador, registro y direcciones de la APK.

```
/home/grupoN/PI2/estacionamiento/
```

Proyecto que se ejecuta para cada grupo: servidor, requirements.txt y .venv. N vale 1, 2, 3 o 4.

```
/etc/systemd/system/
```

Aquí quedan las dos plantillas instaladas: estacionamiento@.service y estacionamiento-tunel@.service.

Dentro de PI2_instalacion:

`direcciones_apk.txt`: las cuatro URL.
`instalacion_*.log`: registro con fecha/hora.
`estado_publico_grupoN.json`: instantánea de la instalación, que puede quedar antigua.

Estos archivos se generan en la Raspberry. No buscar direcciones_apk.txt dentro del ZIP. Además, ~ cambia según la cuenta: para ucu2026 es /home/ucu2026; para grupo1 es /home/grupo1.

E2 Consultar mensajes de servicios

```
sudo journalctl -u estacionamiento@1 -n 30 --no-pager
sudo journalctl -b -u estacionamiento-tunel@1 \
  -n 30 --no-pager
```

Cambiar @1 por @2, @3 o @4. Para problemas de carpeta, copia o comandos, consultar **GUIA_DEL_ZIP.md, apartado 10**.

Alcance: esta ampliación agrega instrucciones de uso y conserva los scripts, el servidor y la APK. No se ejecutó otra instalación. Dominio fijo, MPF-1, sensores, pasarela, almacenamiento persistente y coordenadas relevadas siguen pendientes.

Cuatro grupos: preparar los accesos antes de instalar

Anexo D - Punto de entrada para la práctica independiente del estacionamiento UCU

EL OBJETIVO

1 Una sesión y un proyecto por grupo

Cada grupo entra con su usuario Linux y realiza su propia instalación, programa, prueba y recupera su trabajo. Las cuatro sesiones SSH pueden estar abiertas simultáneamente.

Cada copia tiene código, .venv, archivos y respaldos propios. Su API y su túnel se ejecutan con la identidad de ese grupo.

```
grupo1  ->  puerto 5001
grupo2  ->  puerto 5002
grupo3  ->  puerto 5003
grupo4  ->  puerto 5004
```

2 Carpetas que ya fija el proyecto

```
/home/grupo1/PI2/estacionamiento
/home/grupo2/PI2/estacionamiento
/home/grupo3/PI2/estacionamiento
/home/grupo4/PI2/estacionamiento
```

Seis plazas por desarrollo. Los cuatro proyectos de práctica no representan 24 plazas físicas. La base actual sigue generando datos simulados.

ORDEN DE TRABAJO

3 Docente: preparar una sola vez

Página 7: copiar el ZIP actualizado, ejecutar la preparación común con ucu2026 y revisar cuentas, permisos y Tailscale.

El sistema operativo, la red, SSH, Tailscale, cloudflared y las plantillas se administran desde esa cuenta. Cada PC del curso necesita acceso autorizado a la red privada.

4 Estudiante: trabajar en su cuenta

Página 8: entrar con grupo1, grupo2, grupo3 o grupo4; instalar solo la propia copia y comprobar sus seis plazas.

Página 9: modificar código, operar servicios, consultar la URL actual y conectar la APK a su equipo.

Página 10: respaldar, recuperar y realizar la prueba de independencia con los otros grupos.

Instalación anterior: la preparación nueva conserva los proyectos y servicios existentes. Si ya hay código en la carpeta del grupo, seguir con estado y comprobar.

ALCANCE Y ELECCION DEL RECORRIDO

5 Qué se conserva del PDF y del ZIP

Las páginas 1-5 se mantienen completas, con su numeración original. Describen el recorrido anterior y su evidencia. Este anexo agrega el recorrido desde la cuenta del estudiante.

El instalador general anterior queda disponible para el docente. Para la práctica independiente, usar la preparación de la página 7 y luego la instalación de la página 8.

6 Recursos comunes

CPU, RAM, almacenamiento, sistema y red pertenecen a la misma Raspberry. Los entornos separan el trabajo, sin reservar recursos como cuatro máquinas virtuales.

Los sensores, MPF-1 y pasarela reales siguen pendientes. Su incorporación requerirá acordar como se asignan o distribuyen las lecturas.

Historial: esta versión lo guarda en memoria y lo pierde al reiniciar la API. El respaldo de esta ampliacion conserva código y archivos del proyecto.

Estado de la ampliacion: scripts y guia preparados y probados localmente. Falta ejecutar este recorrido nuevo de extremo a extremo en la Raspberry.

Preparación única del docente

Anexo E - Windows: PowerShell | Raspberry: Bash como ucu2026 | Requiere sistema, SSH y Tailscale preparados

COPIAR LA EDICION ACTUALIZADA

E1 Extraer en una carpeta nueva

En PowerShell de Windows. Si el ZIP tiene otro nombre o Descargas usa otra ruta, ajustar los valores. La carpeta de destino debe ser nueva.

```
$descargas=Join-Path $env:USERPROFILE 'Downloads'
$zip=Join-Path $descargas 'Estacionamiento_UCU_Paquete_Completo.zip'
$carpeta=Join-Path $descargas 'Estacionamiento_UCU_4grupos'
Expand-Archive $zip $carpeta
Test-Path "$carpeta\rpi\preparar_grupos_independientes.sh"
```

El resultado de Test-Path debe ser True. Entrar en la carpeta que contiene rpi y LEEME.md.

E2 Transferir y abrir SSH

```
scp -r "$carpeta\rpi" ucu2026@100.72.228.96:
ssh ucu2026@100.72.228.96
```

Dirección: 100.72.228.96 es la IP comprobada previamente. Usar la IP actual de la Raspberry, obtenida con tailscale ip -4. Verificar la clave del host con el docente.

PREPARAR LOS CUATRO ACCESOS

E3 Ejecutar como administrador

Ya dentro de la Raspberry, en Bash:

```
whoami
tailscale ip -4
bash ~/rpi/preparar_grupos_independientes.sh
```

Debe aparecer ucu2026. El script solicita sudo, instala las dependencias comunes y crea las cuentas que falten. Pide una contraseña por cuenta nueva y conserva las existentes.

Prepara carpetas privadas, registra las plantillas que falten y coloca una copia común de referencia de solo lectura.

Sobre un equipo instalado: conserva código, .venv y servicios activos. Si una plantilla fue personalizada y difiere de la referencia, se detiene para que el docente la revise.

E4 Verificar cuentas y permisos

```
getent passwd grupo1 grupo2 grupo3 grupo4
sudo visudo -cf /etc/sudoers
sudo -l -U grupo1
ls -ld /home/grupo1 /home/grupo1/PI2
```

Repetir la consulta de permisos para grupo2, grupo3 y grupo4. La preparación se detiene si detecta grupos administrativos comunes.

QUE QUEDA INSTALADO

E5 Referencia y control de servicios

```
/usr/local/share/estacionamiento-ucu/
  servidor_estacionamiento.py
  requirements.txt
  mi_grupo.py

/usr/local/sbin/estacionamiento-grupo
/etc/sudoers.d/estacionamiento-grupos
```

La referencia común pertenece a root. Cada estudiante copia su proyecto a su home. El control autorizado deduce el grupo del usuario que invoca sudo.

El permiso se limita a sus dos unidades: estacionamiento@N y estacionamiento-túnel@N. Incluye estado, inicio, parada, reinicio, mensajes y URL.

E6 Entregar el acceso de cada grupo

Comunicar individualmente su cuenta y contraseña. Autorizar su PC en Tailscale y comprobar SSH. Las credenciales no se escriben en el PDF ni en el ZIP.

Equipo nuevo: las plantillas quedan registradas; cada grupo habilita e inicia sus servicios al completar su instalación. Equipo anterior: conserva el estado que ya tenia.

Las pruebas nuevas de permisos y servicios en la Raspberry quedan por realizar. La evidencia anterior del reinicio se conserva como antecedente.

Cada grupo: entrar, instalar y comprobar su copia

Anexo F - Ejemplo grupo1; elegir la cuenta propia | Los comandos Bash se ejecutan dentro de la Raspberry

ENTRAR CON LA CUENTA PROPIA

F1 Elegir una sola linea en PowerShell

```
ssh grupo1@100.72.228.96
ssh grupo2@100.72.228.96
ssh grupo3@100.72.228.96
ssh grupo4@100.72.228.96
```

Usar la IP vigente y la contraseña de su grupo. Al escribir la contraseña no aparecen caracteres. Los cuatro grupos pueden entrar a la vez.

F2 Confirmar identidad en Bash

```
whoami
echo "$HOME"
GESTION_PI2=/usr/local/share/estacionamiento-ucu/mi_grupo.py
```

Para grupo1 deben aparecer grupo1 y /home/grupo1. La variable GESTION_PI2 identifica el programa de gestion y se define otra vez al reconectar.

F3 Instalar en una carpeta nueva

```
python3 "$GESTION_PI2" instalar
python3 "$GESTION_PI2" comprobar
python3 "$GESTION_PI2" estado
```

Copia servidor y requirements.txt; crea .venv e instala las dependencias. Activa únicamente la API y el túnel de la cuenta actual.

RESPETAR EL TRABAJO EXISTENTE

F4 Si el proyecto ya estaba instalado

```
python3 "$GESTION_PI2" estado
python3 "$GESTION_PI2" comprobar
```

El comando instalar rechaza carpetas con contenido y conserva sus archivos. Para usar una copia existente basta con revisar estado y comprobar.

F5 Completar o actualizar dependencias

Si se interrumpio la descarga inicial o se modifico requirements.txt:

```
python3 "$GESTION_PI2" preparar-entorno
python3 "$GESTION_PI2" reiniciar
python3 "$GESTION_PI2" comprobar
```

preparar-entorno usa el código y requirements.txt actuales. Requiere Internet para las dependencias que falten. reiniciar aplica los cambios al proceso propio.

Si comprobar falla durante el arranque: esperar unos segundos y repetir. Consultar estado y mensajes. Un permiso sudo ausente lo corrige el docente con la preparación nueva.

F6 Comprobar los archivos propios

```
cd ~/PI2/estacionamiento
ls -l servidor_estacionamiento.py requirements.txt
.venv/bin/python --version
```

RESULTADO Y PRIMER CAMBIO

F7 Qué valida comprobar

Debe informar OK grupoN, con el número propio. Comprueba seis plazas E001 a E006, simulado=true, lectura vigente de hasta 20 s e historial.

También verifica que su API rechace con HTTP 404 una consulta dirigida a otro grupo.

F8 Programar dentro del proyecto

```
cd ~/PI2/estacionamiento
python3 "$GESTION_PI2" respaldar
nano servidor_estacionamiento.py
.venv/bin/python -m py_compile \
 servidor_estacionamiento.py
python3 "$GESTION_PI2" reiniciar
python3 "$GESTION_PI2" comprobar
```

Modificar solo la copia de su home. Si cambia requirements.txt, completar las dependencias antes de reiniciar.

Puerto ocupado: el servicio puede estar ejecutando ya la API. Usar los comandos de gestion de la página 9 para detener o reiniciar su instancia.

La API actual ofrece JSON. La interfaz visual incluida en el paquete es la APK Android; su configuracion se explica en la página siguiente.

Operar sus servicios y conectar su APK

Anexo G - GESTION_PI2 se define en cada sesión | Las acciones se deducen de la cuenta Linux conectada

COMANDOS DIARIOS

G1 Controlar solo el proyecto propio

```
GESTION_PI2=/usr/local/share/estacionamiento-ucu/mi_grupo.py
python3 "$GESTION_PI2" estado
python3 "$GESTION_PI2" iniciar
python3 "$GESTION_PI2" detener
python3 "$GESTION_PI2" reiniciar
python3 "$GESTION_PI2" mensajes
```

estado: muestra su API y su túnel. Si están detenidos, la orden puede devolver una salida distinta de cero.

iniciar / detener: actuan sobre sus dos servicios. reiniciar: vuelve a cargar la API y recrea su túnel; luego revisar la URL.

mensajes: muestra los últimos 60 mensajes de esos servicios del arranque actual.

G2 Comprobar que cerrar SSH no lo detiene

```
python3 "$GESTION_PI2" comprobar
exit
```

Reconectar con su cuenta, definir GESTION_PI2 otra vez y repetir comprobar sin iniciar el servicio. Las unidades habilitadas también están preparadas para volver al arrancar la Raspberry.

OBTENER LA DIRECCIÓN ACTUAL

G3 Consultar su túnel y probar HTTPS

```
python3 "$GESTION_PI2" url
python3 "$GESTION_PI2" comprobar --publica
```

url imprime solo la dirección base https://...trycloudflare.com. Consulta el journal de la ejecución actual de su túnel.

Si todavía no aparece una URL, esperar unos segundos y repetir. El acceso público requiere salida a Internet y servicios activos.

URL temporal: puede cambiar al recrear el túnel, reiniciar los servicios o la Raspberry. Copiar la URL nueva a la APK después de esas operaciones.

G4 Lista general del docente

Un cambio individual puede dejar antigua la lista global direcciones_apk.txt. Con los cuatro proyectos activos, el docente puede reconstruirla:

```
mkdir -p ~/PI2_instalacion
bash ~/rpi/verificar_despues_reinicio.sh
cat ~/PI2_instalacion/direcciones_apk.txt
```

Este bloque lo ejecuta ucu2026. El verificador general conserva su recorrido original y solo guarda la lista si los cuatro grupos aprueban.

CONFIGURAR EL TELÉFONO

G5 Instalar la APK incluida

```
android/Estaciona-Salto-UCU-0.1.3.apk
```

Transferir el archivo al teléfono e instalarlo si falta. Requiere Android 8 o posterior. La APK se conserva sin recompilar.

G6 Ajustes de su equipo

1. Abrir la aplicacion y entrar a Ajustes.
2. Desactivar Modo demostracion.
3. Pegar la URL base actual del túnel propio.
4. Elegir Equipo 1, 2, 3 o 4.
5. Tocar Guardar y conectar.

Pegar solo https://...trycloudflare.com, sin /api/estado, parámetros ni el puerto 5001-5004.

G7 Probar fuera de la red local

Usar datos móviles, apagar Wi-Fi y desconectar Tailscale en el teléfono. Comprobar las seis plazas y sus cambios; comparar con la prueba HTTPS del grupo.

La consulta pública no necesita cuenta Linux. Al quitar la demo local de la APK, el servidor de esta etapa sigue entregando datos simulados.

Respaldar, recuperar y demostrar independencia

Anexo H - Todo el trabajo se realiza sobre la cuenta propia | Registrar las pruebas reales en clase

CONSERVAR Y RECUPERAR

H1 Crear y listar respaldos propios

```
python3 "$GESTION_PI2" respaldar
python3 "$GESTION_PI2" listar-respaldos
```

Los ZIP quedan en /home/grupoN/PI2/respaldos. Copiar el nombre exacto del respaldo elegido.

Se guardan código y archivos del proyecto. Se excluyen .venv, caches, .git y enlaces. El historial en memoria se pierde al reiniciar y no forma parte del respaldo.

H2 Restaurar una copia del propio grupo

```
python3 "$GESTION_PI2" restaurar \
 NOMBRE_REAL_DEL_RESPALDO.zip
python3 "$GESTION_PI2" comprobar
python3 "$GESTION_PI2" url
```

NOMBRE_REAL_DEL_RESPALDO.zip se reemplaza por el archivo listado, sin carpetas. Se valida grupo, rutas, archivos y sintaxis antes de detener servicios.

Crea además un respaldo actual, conserva la carpeta anterior en ~/PI2/anteriores y recrea .venv. Necesita las dependencias; admite hasta 100 MiB descomprimidos.

Si falla, intenta reponer la carpeta anterior e iniciarla. La copia fallida se conserva. Revisar estado y mensajes.

PRUEBA CON LOS CUATRO GRUPOS

H3 Lista de comprobación en clase

1. Los cuatro entran simultáneamente; todos ejecutan comprobar.

2. Todos guardan un respaldo.

3. grupo1 ejecuta detener. Los otros tres repiten comprobar y deben seguir aprobando.

4. grupo1 ejecuta iniciar, comprobar y url; actualiza la APK si cambio la dirección.

5. grupo1 modifica su código, reinicia y prueba. Los demas comprueban su trabajo.

6. Cerrar y reabrir SSH; comprobar que el servicio continua.

7. Registrar fecha, cuenta, prueba local, HTTPS y APK.

Tarea común: reiniciar toda la Raspberry, cambiar la red o reinstalar el sistema afecta a los cuatro. Lo coordina el docente.

La APK puede conservar datos anteriores al perder conexión. Para demostrar que una API se detuvo, usar las pruebas de API y los mensajes, junto con el estado de la app.

ARCHIVOS Y VALIDACIÓN

H4 Dónde encontrar lo nuevo en el ZIP

```
GUIA_TRABAJO_INDEPENDIENTE.md
COMANDOS_GRUPOS.txt
rpi/preparar_grupos_independientes.sh
rpi/mi_grupo.py
rpi/estacionamiento_grupo_admin.py
pruebas/test_trabajo_independiente.py
```

La guia completa agrega diagnóstico, copia de respaldos a Windows y referencias. Los indices de las páginas 1-5 describen la edicion anterior; el ZIP conserva esos archivos y suma esta ampliacion.

H5 Qué se verifico en esta entrega

14 pruebas locales aprobadas: límite de servicios por usuario, rechazo de identidades/argumentos extra, URL actual, respaldos, restauracion y recuperación ante fallos, y estados separados del simulador.

Las acciones de root y la creacion de .venv se simulan en esas pruebas. La ejecución nueva sobre Raspberry, sudoers, systemd y APK queda pendiente. La evidencia histórica se conserva.

Fuentes: Python venv; systemd.unit; journalctl; sudoers de Debian. Enlaces completos en GUIA_TRABAJO_INDEPENDIENTE.md.